



SAP HANA Cloud, SAP HANA Database Predictive Analysis Library (PAL)

Generated on: 2026-05-24 08:13:35 GMT+0000

SAP HANA Cloud, SAP HANA Database | QRC 1/2026

Public

Original content: https://help.sap.com/docs/HANA_CLOUD_DATABASE/319d36de4fd64ac3afb91b1fb3ce8de?locale=en-US&state=PRODUCTION&version=2026_1_QRC

Warning

This document has been generated from SAP Help Portal and is an incomplete version of the official SAP product documentation. The information included in custom documentation may not reflect the arrangement of topics in SAP Help Portal, and may be missing important aspects and/or correlations to other topics. For this reason, it is not for production use.

For more information, please visit <https://help.sap.com/docs/disclaimer>.

Isolation Forest

This function generates the anomaly score of each sample using the Isolation Forest algorithm.

The algorithm takes advantage of two quantitative properties of anomalies:

- They are the minority consisting of few instances.
- They have attribute-values that are very different from those of normal instances. In other words, anomalies are few and different, which make them more susceptible to a mechanism called Isolation.

We use a binary tree structure called isolation tree (iTree) to recursively separate instances. Observations are isolated by randomly selecting a feature and then randomly selecting a split value between the maximum and minimum values of the selected feature. The number of splittings required to isolate a sample is equivalent to the path length from the root node to the terminating node. Because of the susceptibility to isolation, anomalies are more likely to be isolated closer to the root of an iTree; whereas normal points are more likely to be isolated at the deeper end of an iTree. Isolation Forest (iForest) builds an ensemble of iTrees for a given data set; anomalies are those instances that have short average path lengths on the iTrees.

Using the anomaly score s , we are able to make the following assessment:

- if instances return s very close to 1, then they are definitely anomalies.
- if instances have s much smaller than 0.5, then they are quite safe to be regarded as normal instances.
- if all the instances return $s \approx 0.5$, then the entire sample does not really have any distinct anomaly.

In practice, we assume the proportions of outliers in the training and scoring data sets are the same. Suppose the amount of contamination is 0.25, first we score the training data and compute the 75% quantile of scores. Then we score the scoring data. If the score of a sample is greater than that pre-calculated quantile, the sample can be regarded as an outlier.

Prerequisites

- No missing or null data in the inputs.
- The number of training samples should be at least two.
- The table used to store the tree model is a column table.
- The scoring data (predict) must have an ID column.
- The scoring data should be the same (name, order, data type) as those used in model training.

_SYS_AFL.PAL_ISOLATION_FOREST

Overview of All Tables

Position	Table Name	Parameter Type
1	<DATA table>	IN
2	<PARAMETER table>	IN

Position	Table Name	Parameter Type
3	<MODEL table>	OUT

Input Table

Table	Column	Data Type	Description
DATA	Columns	INTEGER, DOUBLE, DECIMAL(p, s), VARCHAR, or NVARCHAR	The input data.

Parameter Table

Mandatory Parameters

None.

Optional parameters

The following parameters are optional. If a parameter is not specified, PAL will use its default value.

NAME	DATA TYPE	DEFAULT VALUE	DESCRIPTION	DEPENDENCY
THREAD_RATIO	DOUBLE	-1	The ratio of available threads. • 0: single thread • 0~1: percentage • Others: heuristically determined	
SEED	INTEGER	0	Specifies the seed for random number generator. • 0: Uses the current time (in second) as seed • Others: Uses the specified value as seed	
N_ESTIMATORS	INTEGER	100	Specifies the number of trees to grow.	
MAX_SAMPLES	INTEGER	256	Specifies the number of samples to draw from input to train each tree. If MAX_SAMPLES is larger than the number of samples provided, all samples will be used for all trees.	
MAX_FEATURES	INTEGER	0	Specifies the number of features to draw from input to train each tree. 0 means no sampling.	
BOOTSTRAP	INTEGER	0	Specifies sampling method. • 0: Sampling without replacement • 1: Sampling with replacement	
HAS_ID	INTEGER	0	Specifies if the first column of DATA table is used as ID column. • 0: No • 1: Yes	

NAME	DATA TYPE	DEFAULT VALUE	DESCRIPTION	DEPENDENCY
CATEGORICAL_VARIABLE	VARCHAR or NVARCHAR	Detected from input data	Indicates which variables are treated as categorical. The default behavior is: • STRING: categorical • INTEGER and DOUBLE: continuous	Valid only for INTEGER variables; omitted otherwise.

Output Tables

Table	Column	Data Type	Column Name	Description
MODEL	1st Column	INTEGER	TREE_INDEX	Indicates the tree number.
	2nd Column	NCLOB	MODEL_CONTENT	Model content.

Example

Assume that:

- DM_PAL is a schema belonging to USER1;
- USER1 has been assigned the AFL__SYS_AFL_AFLPAL_EXECUTE or AFL__SYS_AFL_AFLPAL_EXECUTE_WITH_GRANT_OPTION role.

```
SET SCHEMA DM_PAL;
```

```
DROP TABLE PAL_IF_DATA_TBL;
CREATE COLUMN TABLE PAL_IF_DATA_TBL (
    "X1" DOUBLE,
    "X2" DOUBLE
);
INSERT INTO PAL_IF_DATA_TBL VALUES (-2, -1);
INSERT INTO PAL_IF_DATA_TBL VALUES (-1, -1);
INSERT INTO PAL_IF_DATA_TBL VALUES (-1, -2);
INSERT INTO PAL_IF_DATA_TBL VALUES (1, 1);
INSERT INTO PAL_IF_DATA_TBL VALUES (1, 2);
INSERT INTO PAL_IF_DATA_TBL VALUES (2, 1);
INSERT INTO PAL_IF_DATA_TBL VALUES (6, 3);
INSERT INTO PAL_IF_DATA_TBL VALUES (-4, 7);
```

```
DROP TABLE PAL_IF_PARAMETER_TBL;
CREATE COLUMN TABLE PAL_IF_PARAMETER_TBL (
    "PARAM_NAME" VARCHAR(256),
    "INT_VALUE" INTEGER,
    "DOUBLE_VALUE" DOUBLE,
    "STRING_VALUE" VARCHAR(100)
);
INSERT INTO PAL_IF_PARAMETER_TBL VALUES ('THREAD_RATIO', NULL, 0, NULL);
INSERT INTO PAL_IF_PARAMETER_TBL VALUES ('SEED', 2, NULL, NULL);
```

```
DROP TABLE PAL_IF_MODEL_TBL;
CREATE COLUMN TABLE PAL_IF_MODEL_TBL ("TREE_INDEX" INTEGER, "MODEL_CONTENT" NCLOB);
```

```
DO BEGIN
    lt_data = SELECT * FROM PAL_IF_DATA_TBL;
    lt_param = SELECT * FROM PAL_IF_PARAMETER_TBL;
    CALL SYS_AFL.PAL_ISOLATION_FOREST (:lt_data, :lt_param, lt_model);
    INSERT INTO PAL_IF_MODEL_TBL SELECT * FROM :lt_model;
END;
```

```
SELECT * FROM PAL_IF_MODEL_TBL;
```

	TREE_INDEX	MODEL_CONTENT
1	0	{"NS":8,"NF":2,"FX":[0,1],"1":{"SF":0,"SV":5.525540351867676,"SS":0},"2":{"
2	1	{"NS":8,"NF":2,"FX":[0,1],"1":{"SF":0,"SV":5.525540351867676,"SS":0},"2":{"
3	2	{"NS":8,"NF":2,"FX":[1,0],"1":{"SF":0,"SV":4.684035301208496,"SS":0},"2":{"
4	3	{"NS":8,"NF":2,"FX":[1,0],"1":{"SF":0,"SV":4.684035301208496,"SS":0},"2":{"
5	4	{"NS":8,"NF":2,"FX":[1,0],"1":{"SF":0,"SV":5.334948539733887,"SS":0},"2":{"
6	5	{"NS":8,"NF":2,"FX":[1,0],"1":{"SF":0,"SV":5.334948539733887,"SS":0},"2":{"
7	6	{"NS":8,"NF":2,"FX":[0,1],"1":{"SF":0,"SV":3.6340508460998537,"SS":0},"2"
8	7	{"NS":8,"NF":2,"FX":[0,1],"1":{"SF":0,"SV":3.6340508460998537,"SS":0},"2"
9	8	{"NS":8,"NF":2,"FX":[1,0],"1":{"SF":0,"SV":-0.6070070266723633,"SS":0},"2
10	9	{"NS":8,"NF":2,"FX":[1,0],"1":{"SF":0,"SV":-0.6070070266723633,"SS":0},"2
11	10	{"NS":8,"NF":2,"FX":[0,1],"1":{"SF":1,"SV":5.90471076965332,"SS":0},"2":{"
12	11	{"NS":8,"NF":2,"FX":[0,1],"1":{"SF":1,"SV":5.90471076965332,"SS":0},"2":{"
13	12	{"NS":8,"NF":2,"FX":[1,0],"1":{"SF":1,"SV":5.519649028778076,"SS":0},"2":{"
14	13	{"NS":8,"NF":2,"FX":[1,0],"1":{"SF":1,"SV":5.519649028778076,"SS":0},"2":{"
15	14	{"NS":8,"NF":2,"FX":[0,1],"1":{"SF":0,"SV":-0.4324929714202881,"SS":0},"2
16	15	{"NS":8,"NF":2,"FX":[0,1],"1":{"SF":0,"SV":-0.4324929714202881,"SS":0},"2

_SYS_AFL.PAL_ISOLATION_FOREST_PREDICT

Overview of all tables

Position	Table Name	Parameter Type
1	<DATA table>	IN
2	<MODEL table>	IN
3	<PARAMETER table>	IN
4	<RESULT table>	OUT

Input Table

Table	Column	Reference for Output Table	Data Type	Description	Constraint
DATA	1st Column	ID	INTEGER, VARCHAR, or NVARCHAR	Data ID	This must be the first column.
	Other Columns		INTEGER, DOUBLE, DECIMAL(p, s), VARCHAR, or NVARCHAR	The input features.	
MODEL	1st Column		INTEGER	Tree index.	
	2nd Column		NCLOB	Model content.	

Parameter Table**Mandatory Parameters**

None.

Optional parameters

The following parameters are optional. If a parameter is not specified, PAL will use its default value.

NAME	DATA TYPE	DEFAULT VALUE	DESCRIPTION	DEPENDENCY
THREAD_RATIO	DOUBLE	-1	The ratio of available threads. 0: single thread 0~1: percentage - Others: heuristically determined	
CONTAMINATION	DOUBLE	0.1	The proportion of outliers in the data set. CONTAMINATION should be in the range (0, 0.5].	

Output table

Table	Column	Data Type	Column Name	Description	Constraint
RESULT	1st Column	ID column type	ID	ID column name.	Must be the first column.
	2nd Column	DOUBLE	SCORE	Scoring result.	
	3rd Column	INTEGER	LABEL	-1 for outliers and 1 for inliers.	

Examples

Assume that:

- DM_PAL is a schema belonging to USER1; and
- USER1 has been assigned the AFL__SYS_AFL_AFLPAL_EXECUTE or AFL__SYS_AFL_AFLPAL_EXECUTE_WITH_GRANT_OPTION role.

```
SET SCHEMA DM_PAL;
```

```
DROP TABLE PAL_IF_SCORING_DATA_TBL;
CREATE COLUMN TABLE PAL_IF_SCORING_DATA_TBL (
  "ID" INTEGER,
  "X1" DOUBLE,
  "X2" DOUBLE
);
INSERT INTO PAL_IF_SCORING_DATA_TBL VALUES (0, -2, -1);
INSERT INTO PAL_IF_SCORING_DATA_TBL VALUES (1, -1, -1);
INSERT INTO PAL_IF_SCORING_DATA_TBL VALUES (2, -1, -2);
INSERT INTO PAL_IF_SCORING_DATA_TBL VALUES (3, 1, 1);
INSERT INTO PAL_IF_SCORING_DATA_TBL VALUES (4, 1, 2);
INSERT INTO PAL_IF_SCORING_DATA_TBL VALUES (5, 2, 1);
INSERT INTO PAL_IF_SCORING_DATA_TBL VALUES (6, 6, 3);
INSERT INTO PAL_IF_SCORING_DATA_TBL VALUES (7, -4, 7);
```

```
DROP TABLE PAL_IF_SCORING_PARAMETER_TBL;
```

```
CREATE COLUMN TABLE PAL_IF_SCORING_PARAMETER_TBL (
    "PARAM_NAME" VARCHAR(256),
    "INT_VALUE" INTEGER,
    "DOUBLE_VALUE" DOUBLE,
    "STRING_VALUE" VARCHAR(100)
);
INSERT INTO PAL_IF_SCORING_PARAMETER_TBL VALUES ('THREAD_RATIO', NULL, 0, NULL);
INSERT INTO PAL_IF_SCORING_PARAMETER_TBL VALUES ('CONTAMINATION', NULL, 0.25, NULL);

DROP TABLE PAL_IF_RESULT_TBL;
CREATE COLUMN TABLE PAL_IF_RESULT_TBL ("ID" INTEGER, "SCORE" DOUBLE, "LABEL" INTEGER);

DO BEGIN
    lt_data = SELECT * FROM PAL_IF_SCORING_DATA_TBL;
    lt_model = SELECT * FROM PAL_IF_MODEL_TBL;
    lt_param = SELECT * FROM PAL_IF_SCORING_PARAMETER_TBL;
    CALL _SYS_AFL.PAL_ISOLATION_FOREST_PREDICT (:lt_data, :lt_model, :lt_param, lt_result);
    INSERT INTO PAL_IF_RESULT_TBL SELECT * FROM :lt_result;
END;

SELECT * FROM PAL_IF_RESULT_TBL;
```

Expected Result

	ID	SCORE	LABEL
1	0	0.44689698428888736	1
2	1	0.411048431708785	1
3	2	0.4989311315568398	1
4	3	0.40779644687145056	1
5	4	0.42326365421687284	1
6	5	0.4432695217396772	1
7	6	0.6195125059059307	-1
8	7	0.6388737067483532	-1

_SYS_AFL.PAL_ISOLATION_FOREST_EXPLAIN

Overview of all tables

Position	Table Name	Parameter Type
1	<DATA table>	IN
2	<MODEL table>	IN
3	<PARAMETER table>	IN
4	<RESULT table>	OUT

Input Table

Table	Column	Reference for Output Table	Data Type	Description	Constraint
DATA	1st Column	ID	INTEGER, VARCHAR, or NVARCHAR	Data ID	This must be the first column.

Table	Column	Reference for Output Table	Data Type	Description	Constraint
	Other Columns		INTEGER, DOUBLE, or DECIMAL(p, s), VARCHAR, or NVARCHAR	The input features	
MODEL	1st Column		INTEGER	Tree index	
	2nd Column		NCLOB	Model content	

Parameter Table

Mandatory Parameters

None.

Optional parameters

The following parameters are optional. If a parameter is not specified, PAL uses its default value.

NAME	DATA TYPE	DEFAULT VALUE	DESCRIPTION	DEPENDENCY
THREAD_RATIO	DOUBLE	-1	Specifies the ratio of available threads: 0: single thread 0~1: percentage - Others: heuristically determined	
CONTAMINATION	DOUBLE	0.1	The proportion of outliers in the data set. Must be within the range (0, 0.5].	
EXPLAIN_SCOPE	INTEGER	0	Defines the scope of explanation. 0: Only outliers 1: All samples	
TOP_K_ATTRIBUTIONS	INTEGER	10	Specifies the number (k) of key features to output that have the most contribution to the model's predictions or outcomes.	

Output table

Table	Column	Data Type	Column Name	Description	Constraint
RESULT	1st Column	ID column type	ID	ID column name	Must be the first column.
	2nd Column	DOUBLE	SCORE	Scoring result	
	3rd Column	INTEGER	LABEL	-1 for outliers and 1 for inliers	
	4th Column	NCLOB	REASON_CODE	Attributions of features	

Example

Assume that:

- DM_PAL is a schema belonging to USER1; and
- USER1 has been assigned the AFL__SYS_AFL_AFLPAL_EXECUTE or AFL__SYS_AFL_AFLPAL_EXECUTE_WITH_GRANT_OPTION role.

```
SET SCHEMA DM_PAL;
```

```
DROP TABLE PAL_IF_EXPLAINING_DATA_TBL;
```

```
CREATE COLUMN TABLE PAL_IF_EXPLAINING_DATA_TBL (
```

```
    "ID" INTEGER,
```

```
    "X1" DOUBLE,
```

```
    "X2" DOUBLE
```

```
);
```

```
INSERT INTO PAL_IF_EXPLAINING_DATA_TBL VALUES (0, -2, -1);
```

```
INSERT INTO PAL_IF_EXPLAINING_DATA_TBL VALUES (1, -1, -1);
```

```
INSERT INTO PAL_IF_EXPLAINING_DATA_TBL VALUES (2, -1, -2);
```

```
INSERT INTO PAL_IF_EXPLAINING_DATA_TBL VALUES (3, 1, 1);
```

```
INSERT INTO PAL_IF_EXPLAINING_DATA_TBL VALUES (4, 1, 2);
```

```
INSERT INTO PAL_IF_EXPLAINING_DATA_TBL VALUES (5, 2, 1);
```

```
INSERT INTO PAL_IF_EXPLAINING_DATA_TBL VALUES (6, 6, 3);
```

```
INSERT INTO PAL_IF_EXPLAINING_DATA_TBL VALUES (7, -4, 7);
```

```
DROP TABLE PAL_IF_EXPLAINING_PARAMETER_TBL;
```

```
CREATE COLUMN TABLE PAL_IF_EXPLAINING_PARAMETER_TBL (
```

```
    "PARAM_NAME" VARCHAR(256),
```

```
    "INT_VALUE" INTEGER,
```

```
    "DOUBLE_VALUE" DOUBLE,
```

```
    "STRING_VALUE" VARCHAR(100)
```

```
);
```

```
INSERT INTO PAL_IF_EXPLAINING_PARAMETER_TBL VALUES ('CONTAMINATION', NULL, 0.25, NULL);
```

```
INSERT INTO PAL_IF_EXPLAINING_PARAMETER_TBL VALUES ('EXPLAIN_SCOPE', 0, NULL, NULL);
```

```
INSERT INTO PAL_IF_EXPLAINING_PARAMETER_TBL VALUES ('TOP_K_ATTRIBUTIONS', 10, NULL, NULL);
```

```
DROP TABLE PAL_IF_EXPLAINING_RESULT_TBL;
```

```
CREATE COLUMN TABLE PAL_IF_EXPLAINING_RESULT_TBL ("ID" INTEGER, "SCORE" DOUBLE, "LABEL" INTEGER, "REASON_CODE"
```

```
DO BEGIN
```

```
    lt_data = SELECT * FROM PAL_IF_EXPLAINING_DATA_TBL;
```

```
    lt_model = SELECT * FROM PAL_IF_MODEL_TBL;
```

```
    lt_param = SELECT * FROM PAL_IF_EXPLAINING_PARAMETER_TBL;
```

```
    CALL _SYS_AFL.PAL_ISOLATION_FOREST_EXPLAIN (:lt_data, :lt_model, :lt_param, lt_result);
```

```
    INSERT INTO PAL_IF_EXPLAINING_RESULT_TBL SELECT * FROM :lt_result;
```

```
END;
```

```
SELECT * FROM PAL_IF_EXPLAINING_RESULT_TBL;
```

Expected Result

	ID ▾	SCORE ▾	LABEL ▾	REASON_CODE
1	0	0.44689698428888736	1	NULL
2	1	0.411048431708785	1	NULL
3	2	0.4989311315568398	1	NULL
4	3	0.40779644687145056	1	NULL
5	4	0.42326365421687284	1	NULL
6	5	0.4432695217396772	1	NULL
7	6	0.6195125059059307	-1	[{"attr": "X1", "val": -0.589814090727926, "pct": 75.6}, {"attr": "X2", "val": -0.18
8	7	0.6388737067483532	-1	[{"attr": "X1", "val": -0.3155324209584644, "pct": 75.3}, {"attr": "X2", "val": -0.1

_SYS_AFL.PAL_MASSIVE_ISOLATION_FOREST

This function provides the massive mode for _SYS_AFL.PAL_ISOLATION_FOREST. The massive mode can process multiple datasets and parameters in parallel by specifying an additional Group_ID column.

For detailed information on the massive execution mechanism, see [Shared Parameters](#) and [Error Handling](#).

Overview of All Tables

Position	Table Name	Parameter Type
1	<DATA table>	IN
2	<PARAMETER table>	IN
3	<MODEL table>	OUT
4	<ErrorMsgTBL table>	OUT

Input Table

Table	Column	Data Type	Description
DATA	1st column	INTEGER, VARCHAR or NVARCHAR	Group ID
	Other columns	INTEGER, DOUBLE, DECIMAL(p, s), VARCHAR or NVARCHAR	The input data.
PARAMETER	1st column	INTEGER, VARCHAR or NVARCHAR	Group ID
	2nd column	VARCHAR or NVARCHAR	Parameter name
	3rd column	INTEGER	Integer value
	4th column	DOUBLE	Double value
	5th column	VARCHAR or NVARCHAR	String value

Parameter Table

See _SYS_AFL.PAL_ISOLATION_FOREST.

Output Table

Table	Column	Data Type	Column Name	Description
MODEL	1st column	Group ID (in input table) column type	GROUP_ID	Group ID
	2nd column	INTEGER	TREE_INDEX	Indicates the tree number.
	3rd column	NCLOB	MODEL_CONTENT	Model content.

Table	Column	Data Type	Column Name	Description
ERRORMsgTBL	1st column	Group ID (in input table) column type	GROUP_ID	Group ID
	2nd column	VARCHAR, NVARCHAR	ERROR_TIMESTAMP	Error time stamp
	3rd column	INTEGER	ERRORCODE	Error code

Table	Column	Data Type	Column Name	Description
	4th column	VARCHAR, NVARCHAR	MESSAGE	Error message

Examples

Assume that:

- DM_PAL is a schema belonging to USER1;
- USER1 has been assigned the AFL__SYS_AFL_AFLPAL_EXECUTE or AFL__SYS_AFL_AFLPAL_EXECUTE_WITH_GRANT_OPTION role.

```
SET SCHEMA DM_PAL;
```

```
DROP TABLE PAL_MASSIVE_IF_DATA_TBL;
CREATE COLUMN TABLE PAL_MASSIVE_IF_DATA_TBL (
    "GroupID" INTEGER,
    "X1" DOUBLE,
    "X2" DOUBLE
);
INSERT INTO PAL_MASSIVE_IF_DATA_TBL VALUES (0, -2, -1);
INSERT INTO PAL_MASSIVE_IF_DATA_TBL VALUES (0, -1, -1);
INSERT INTO PAL_MASSIVE_IF_DATA_TBL VALUES (0, -1, -2);
INSERT INTO PAL_MASSIVE_IF_DATA_TBL VALUES (0, 1, 1);
INSERT INTO PAL_MASSIVE_IF_DATA_TBL VALUES (0, 1, 2);
INSERT INTO PAL_MASSIVE_IF_DATA_TBL VALUES (0, 2, 1);
INSERT INTO PAL_MASSIVE_IF_DATA_TBL VALUES (0, 6, 3);
INSERT INTO PAL_MASSIVE_IF_DATA_TBL VALUES (0, -4, 7);
INSERT INTO PAL_MASSIVE_IF_DATA_TBL VALUES (1, 0.3, -0.3);
INSERT INTO PAL_MASSIVE_IF_DATA_TBL VALUES (1, 0.5, -0.5);
INSERT INTO PAL_MASSIVE_IF_DATA_TBL VALUES (1, 100, -100);
```

```
DROP TABLE PAL_MASSIVE_IF_PARAMETER_TBL;
CREATE COLUMN TABLE PAL_MASSIVE_IF_PARAMETER_TBL (
    "GroupID" INTEGER,
    "PARAM_NAME" VARCHAR(256),
    "INT_VALUE" INTEGER,
    "DOUBLE_VALUE" DOUBLE,
    "STRING_VALUE" VARCHAR(100)
);
INSERT INTO PAL_MASSIVE_IF_PARAMETER_TBL VALUES (0, 'SEED', 2, NULL, NULL);
INSERT INTO PAL_MASSIVE_IF_PARAMETER_TBL VALUES (1, 'MAX_FEATURES', 1, NULL, NULL);
INSERT INTO PAL_MASSIVE_IF_PARAMETER_TBL VALUES (1, 'SEED', 2, NULL, NULL);
```

```
DROP TABLE PAL_MASSIVE_IF_MODEL_TBL;
CREATE COLUMN TABLE PAL_MASSIVE_IF_MODEL_TBL ("GroupID" INTEGER, "TREE_INDEX" INTEGER, "MODEL_CONTENT"
```

```
DROP TABLE PAL_MASSIVE_ERRORMSG_TBL;
CREATE COLUMN TABLE PAL_MASSIVE_ERRORMSG_TBL ("GroupID" INTEGER, "ERROR_TIMESTAMP" NVARCHAR(50), "ERROR"
```

```
DO BEGIN
    lt_data = SELECT * FROM PAL_MASSIVE_IF_DATA_TBL;
    lt_param = SELECT * FROM PAL_MASSIVE_IF_PARAMETER_TBL;
    CALL _SYS_AFL.PAL_MASSIVE_ISOLATION_FOREST (:lt_data, :lt_param, lt_model, lt_errmsg);
    INSERT INTO PAL_MASSIVE_IF_MODEL_TBL SELECT * FROM :lt_model;
    INSERT INTO PAL_MASSIVE_ERRORMSG_TBL SELECT * FROM :lt_errmsg;
END;
```

```
SELECT * FROM PAL_MASSIVE_IF_MODEL_TBL;
SELECT * FROM PAL_MASSIVE_ERRORMSG_TBL;
```

Expected Result

	GroupID ▾	TREE_INDEX ▾	MODEL_CONTENT ▾
1	1	0	{"NS":3,"NF":1,"FX":[1],"1":{"SF":1,"SV":-95.71245193220675,"SS":0},"2":{"S...
2	1	1	{"NS":3,"NF":1,"FX":[1],"1":{"SF":1,"SV":-95.71245193220675,"SS":0},"2":{"S...
3	1	2	{"NS":3,"NF":1,"FX":[1],"1":{"SF":1,"SV":-9.86578193306923,"SS":0},"2":{"SF...
4	1	3	{"NS":3,"NF":1,"FX":[1],"1":{"SF":1,"SV":-9.86578193306923,"SS":0},"2":{"SF...
5	1	4	{"NS":3,"NF":1,"FX":[0],"1":{"SF":0,"SV":91.52424825429917,"SS":0},"2":{"SF...
6	1	5	{"NS":3,"NF":1,"FX":[0],"1":{"SF":0,"SV":91.52424825429917,"SS":0},"2":{"SF...
7	1	6	{"NS":3,"NF":1,"FX":[1],"1":{"SF":1,"SV":-64.50909024775028,"SS":0},"2":{"S...
8	1	7	{"NS":3,"NF":1,"FX":[1],"1":{"SF":1,"SV":-64.50909024775028,"SS":0},"2":{"S...
9	1	8	{"NS":3,"NF":1,"FX":[0],"1":{"SF":0,"SV":80.03958841562272,"SS":0},"2":{"SF...

_SYS_AFL.PAL_MASSIVE_ISOLATION_FOREST_PREDICT

This function provides the massive mode for _SYS_AFL . PAL_ISOLATION_FOREST_PREDICT. The massive mode can process multiple datasets and parameters in parallel by specifying an additional Group_ID column.

For detailed information on the massive execution mechanism, see [Shared Parameters](#) and [Error Handling](#).

Overview of All Tables

Position	Table Name	Parameter Type
1	<DATA table>	IN
2	<MODEL table>	IN
3	<PARAMETER table>	IN
4	<RESULT table>	OUT
5	<ErrorMsgTBL table>	OUT

Input Table

Table	Column	Reference for Output Table	Data Type	Description
DATA	1st column		INTEGER, VARCHAR or NVARCHAR	Group ID
	2nd column	ID	INTEGER, VARCHAR, or NVARCHAR	Data ID
	Other columns		INTEGER, DOUBLE, DECIMAL(p, s), VARCHAR or NVARCHAR	The input features.
MODEL	1st column		INTEGER, VARCHAR or NVARCHAR	Group ID
	2nd column		INTEGER	Tree index.
	3rd column		NCLOB	Model content.
PARAMETER	1st column		INTEGER, VARCHAR or NVARCHAR	Group ID
	2nd column		VARCHAR or NVARCHAR	Parameter name

Table	Column	Reference for Output Table	Data Type	Description
	3rd column		INTEGER	Integer value
	4th column		DOUBLE	Double value
	5th column		VARCHAR or NVARCHAR	String value

Parameter Table

See _SYS_AFL.PAL_ISOLATION_FOREST_PREDICT.

Output Table

Table	Column	Data Type	Column Name	Description
RESULT	1st column	Group ID (in input table) column type	GROUP_ID	Group ID
	2nd column	ID column type	ID	ID column name.
	3rd column	DOUBLE	SCORE	Scoring result.
	4th column	INTEGER	LABEL	-1 for outliers and 1 for inliers.

Table	Column	Data Type	Column Name	Description
ERRORMsgTBL	1st column	Group ID (in input table) column type	GROUP_ID	Group ID
	2nd column	VARCHAR, NVARCHAR	ERROR_TIMESTAMP	Error time stamp
	3rd column	INTEGER	ERRORCODE	Error code
	4th column	VARCHAR, NVARCHAR	MESSAGE	Error message

Examples

Assume that:

- DM_PAL is a schema belonging to USER1;
- USER1 has been assigned the AFL__SYS_AFL_AFLPAL_EXECUTE or AFL__SYS_AFL_AFLPAL_EXECUTE_WITH_GRANT_OPTION role.

```
SET SCHEMA DM_PAL;
```

```
DROP TABLE PAL_MASSIVE_IF_SCORING_DATA_TBL;
CREATE COLUMN TABLE PAL_MASSIVE_IF_SCORING_DATA_TBL (
    "GroupID" INTEGER,
    "ID" INTEGER,
    "X1" DOUBLE,
    "X2" DOUBLE
);
INSERT INTO PAL_MASSIVE_IF_SCORING_DATA_TBL VALUES (0, 0, -2, -1);
INSERT INTO PAL_MASSIVE_IF_SCORING_DATA_TBL VALUES (0, 1, -1, -1);
INSERT INTO PAL_MASSIVE_IF_SCORING_DATA_TBL VALUES (0, 2, -1, -2);
INSERT INTO PAL_MASSIVE_IF_SCORING_DATA_TBL VALUES (0, 3, 6, 3);
INSERT INTO PAL_MASSIVE_IF_SCORING_DATA_TBL VALUES (0, 4, -4, 7);
INSERT INTO PAL_MASSIVE_IF_SCORING_DATA_TBL VALUES (1, 0, 0.1, -0.1);
INSERT INTO PAL_MASSIVE_IF_SCORING_DATA_TBL VALUES (1, 1, 0, 0);
```

```

INSERT INTO PAL_MASSIVE_IF_SCORING_DATA_TBL VALUES (1, 2, 90, -90);

DROP TABLE PAL_MASSIVE_IF_SCORING_PARAMETER_TBL;
CREATE COLUMN TABLE PAL_MASSIVE_IF_SCORING_PARAMETER_TBL (
    "GroupID" INTEGER,
    "PARAM_NAME" VARCHAR(256),
    "INT_VALUE" INTEGER,
    "DOUBLE_VALUE" DOUBLE,
    "STRING_VALUE" VARCHAR(100)
);
INSERT INTO PAL_MASSIVE_IF_SCORING_PARAMETER_TBL VALUES (0, 'CONTAMINATION', NULL, 0.4, NULL);
INSERT INTO PAL_MASSIVE_IF_SCORING_PARAMETER_TBL VALUES (1, 'CONTAMINATION', NULL, 0.33, NULL);

DROP TABLE PAL_MASSIVE_IF_RESULT_TBL;
CREATE COLUMN TABLE PAL_MASSIVE_IF_RESULT_TBL ("GroupID" INTEGER, "ID" INTEGER, "SCORE" DOUBLE, "LABEL" VARCHAR(10));

DROP TABLE PAL_MASSIVE_ERRORMSG_TBL;
CREATE COLUMN TABLE PAL_MASSIVE_ERRORMSG_TBL ("GroupID" INTEGER, "ERROR_TIMESTAMP" NVARCHAR(50), "ERROR_MESSAGE" NVARCHAR(255));

DO BEGIN
    lt_data = SELECT * FROM PAL_MASSIVE_IF_SCORING_DATA_TBL;
    lt_model = SELECT * FROM PAL_MASSIVE_IF_MODEL_TBL;
    lt_param = SELECT * FROM PAL_MASSIVE_IF_SCORING_PARAMETER_TBL;
    CALL _SYS_AFL.PAL_MASSIVE_ISOLATION_FOREST_PREDICT (:lt_data, :lt_model, :lt_param, lt_result, lt_errmsg);
    INSERT INTO PAL_MASSIVE_IF_RESULT_TBL SELECT * FROM :lt_result;
    INSERT INTO PAL_MASSIVE_ERRORMSG_TBL SELECT * FROM :lt_errmsg;
END;

SELECT * FROM PAL_MASSIVE_IF_RESULT_TBL;
SELECT * FROM PAL_MASSIVE_ERRORMSG_TBL;

```

Expected Result

	GroupID	ID	SCORE	LABEL
1	1	0	0.3172160416197904	1
2	1	1	0.3172160416197904	1
3	1	2	0.5257253020856127	-1
4	0	0	0.4468969842888874	1
5	0	1	0.411048431708785	1
6	0	2	0.4989311315568398	1
7	0	3	0.6195125059059307	-1
8	0	4	0.6388737067483532	-1

_SYS_AFL.PAL_MASSIVE_ISOLATION_FOREST_EXPLAIN

This function provides a massive mode for _SYS_AFL.PAL_ISOLATION_FOREST_EXPLAIN. It can process multiple datasets and parameters in parallel by specifying an additional Group_ID column.

For detailed information on the massive execution mechanism, see [Shared Parameters](#) and [Error Handling](#).

Overview of All Tables

Position	Table Name	Parameter Type
1	<DATA table>	IN
2	<MODEL table>	IN
3	<PARAMETER table>	IN

Position	Table Name	Parameter Type
4	<RESULT table>	OUT
5	<ErrorMsgTBL table>	OUT

Input Table

Table	Column	Reference for Output Table	Data Type	Description
DATA	1st column		INTEGER, VARCHAR or NVARCHAR	Group ID
	2nd column	ID	INTEGER, VARCHAR, or NVARCHAR	Data ID
	Other columns		INTEGER, DOUBLE, DECIMAL(p, s), VARCHAR, or NVARCHAR	The input features
MODEL	1st column		INTEGER, VARCHAR or NVARCHAR	Group ID
	2nd column		INTEGER	Tree index
	3rd column		NCLOB	Model content
PARAMETER	1st column		INTEGER, VARCHAR or NVARCHAR	Group ID
	2nd column		VARCHAR or NVARCHAR	Parameter name
	3rd column		INTEGER	Integer value
	4th column		DOUBLE	Double value
	5th column		VARCHAR or NVARCHAR	String value

Parameter Table

See the parameters in _SYS_AFL.PAL_ISOLATION_FOREST_EXPLAIN.

Output Tables

Table	Column	Data Type	Column Name	Description
RESULT	1st column	Group ID (in input table) column type	GROUP_ID	Group ID
	2nd column	ID column type	ID	ID column name
	3rd column	DOUBLE	SCORE	Scoring result
	4th column	INTEGER	LABEL	-1 for outliers and 1 for inliers
	5th column	NCLOB	REASON_CODE	Attributions of features

Table	Column	Data Type	Column Name	Description
ERRORMsgTBL	1st column	Group ID (in input table) column type	GROUP_ID	Group ID
	2nd column	VARCHAR, NVARCHAR	ERROR_TIMESTAMP	Error time stamp

Table	Column	Data Type	Column Name	Description
	3rd column	INTEGER	ERRORCODE	Error code
	4th column	VARCHAR, NVARCHAR	MESSAGE	Error message

Examples

Assume that:

- DM_PAL is a schema belonging to USER1;
- USER1 has been assigned the AFL__SYS_AFL_AFLPAL_EXECUTE or AFL__SYS_AFL_AFLPAL_EXECUTE_WITH_GRANT_OPTION role.

```
SET SCHEMA DM_PAL;
```

```
DROP TABLE PAL_MASSIVE_IF_EXPLAINING_DATA_TBL;
CREATE COLUMN TABLE PAL_MASSIVE_IF_EXPLAINING_DATA_TBL (
    "GroupID" INTEGER,
    "ID" INTEGER,
    "X1" DOUBLE,
    "X2" DOUBLE
);
INSERT INTO PAL_MASSIVE_IF_EXPLAINING_DATA_TBL VALUES (0, 0, -2, -1);
INSERT INTO PAL_MASSIVE_IF_EXPLAINING_DATA_TBL VALUES (0, 1, -1, -1);
INSERT INTO PAL_MASSIVE_IF_EXPLAINING_DATA_TBL VALUES (0, 2, -1, -2);
INSERT INTO PAL_MASSIVE_IF_EXPLAINING_DATA_TBL VALUES (0, 3, 6, 3);
INSERT INTO PAL_MASSIVE_IF_EXPLAINING_DATA_TBL VALUES (0, 4, -4, 7);
INSERT INTO PAL_MASSIVE_IF_EXPLAINING_DATA_TBL VALUES (1, 0, 0.1, -0.1);
INSERT INTO PAL_MASSIVE_IF_EXPLAINING_DATA_TBL VALUES (1, 1, 0, 0);
INSERT INTO PAL_MASSIVE_IF_EXPLAINING_DATA_TBL VALUES (1, 2, 90, -90);

DROP TABLE PAL_MASSIVE_IF_EXPLAINING_PARAMETER_TBL;
CREATE COLUMN TABLE PAL_MASSIVE_IF_EXPLAINING_PARAMETER_TBL (
    "GroupID" INTEGER,
    "PARAM_NAME" VARCHAR(256),
    "INT_VALUE" INTEGER,
    "DOUBLE_VALUE" DOUBLE,
    "STRING_VALUE" VARCHAR(100)
);
INSERT INTO PAL_MASSIVE_IF_EXPLAINING_PARAMETER_TBL VALUES (0, 'CONTAMINATION', NULL, 0.4, NULL);
INSERT INTO PAL_MASSIVE_IF_EXPLAINING_PARAMETER_TBL VALUES (1, 'CONTAMINATION', NULL, 0.33, NULL);
INSERT INTO PAL_MASSIVE_IF_EXPLAINING_PARAMETER_TBL VALUES (1, 'EXPLAIN_SCOPE', 1, NULL, NULL);

DROP TABLE PAL_MASSIVE_IF_EXPLAINING_RESULT_TBL;
CREATE COLUMN TABLE PAL_MASSIVE_IF_EXPLAINING_RESULT_TBL ("GroupID" INTEGER, "ID" INTEGER, "SCORE"

DROP TABLE PAL_MASSIVE_ERRORMSG_TBL;
CREATE COLUMN TABLE PAL_MASSIVE_ERRORMSG_TBL ("GroupID" INTEGER, "ERROR_TIMESTAMP" NVARCHAR(50), "I

DO BEGIN
    lt_data = SELECT * FROM PAL_MASSIVE_IF_EXPLAINING_DATA_TBL;
    lt_model = SELECT * FROM PAL_MASSIVE_IF_MODEL_TBL;
    lt_param = SELECT * FROM PAL_MASSIVE_IF_EXPLAINING_PARAMETER_TBL;
    CALL SYS_AFL.PAL_MASSIVE_ISOLATION_FOREST_EXPLAIN (:lt_data, :lt_model, :lt_param, lt_result, lt
    INSERT INTO PAL_MASSIVE_IF_EXPLAINING_RESULT_TBL SELECT * FROM :lt_result;
    INSERT INTO PAL_MASSIVE_ERRORMSG_TBL SELECT * FROM :lt_errmsg;
END;

SELECT * FROM PAL_MASSIVE_IF_EXPLAINING_RESULT_TBL;
SELECT * FROM PAL_MASSIVE_ERRORMSG_TBL;
```

Expected Result

	GroupID ▾	ID ▾	SCORE ▾	LABEL ▾	REASON_CC
1	0	0	0.44689698428888736	1	NULL
2	0	1	0.411048431708785	1	NULL
3	0	2	0.4989311315568398	1	NULL
4	0	3	0.6195125059059307	-1	[{"attr":"X1","val":-0.589814090727926,"pct":75.6}, {"attr":"X2","v
5	0	4	0.6388737067483532	-1	[{"attr":"X1","val":-0.3155324209584644,"pct":75.3}, {"attr":"X2","
6	1	0	0.3172160416197904	1	[{"attr":"X2","val":0.1866666666666667,"pct":100.0}, {"attr":"X1",'
7	1	1	0.3172160416197904	1	[{"attr":"X2","val":0.1866666666666667,"pct":100.0}, {"attr":"X1",'
8	1	2	0.5257253020856127	-1	[{"attr":"X2","val":-0.29333333333333297,"pct":100.0}, {"attr":"X1